CONFIDENTIAL — TECHNICAL HANDOVER DOCUMENT

DKJU Platform
Code Architecture & Handover Documentation

Madrasa Management & Multi-Level Competition (التنافس) System
Next.js 15 · React 19 · Firebase (Firestore / Auth / Storage / Functions)

Document purpose: Codebase handover reference for engineering leadership review prior to client handover.

Scope: Full repository — all portals, pages, API routes, data model, environment configuration, and security posture.

Prepared: August 2026

Live users: Yes — this system serves active production users. Treat all recommendations in this document as pre-handover priorities.

INTERNAL USE — FOR CTO REVIEW PRIOR TO CLIENT TRANSFER

Table of Contents

1 Executive Summary

2 Technology Stack

3 Repository Structure

4 Application Architecture

- 4.1 Rendering & Routing Model
- 4.2 Authentication & Session Architecture
- 4.3 Portal Map & Role Resolution (with codebase discrepancy notes)
- 4.4 Layout, Navigation & UI Composition

5 Complete Portal & Page Inventory

6 API Route Inventory

7 Data Model — Firestore Collections

8 Firebase Cloud Functions

9 External Services & Third-Party Dependencies

10 Environment Variables Reference

11 Security Implementation & Findings

12 PWA, Offline Support & Deployment

1. Executive Summary

DKJU is a full-stack madrasa (Islamic school) management platform serving a multi-tier institutional hierarchy in Kerala, India: **State → District → Mekhala (sub-district) → Sadar (individual madrasa)**. It manages student admissions, attendance, examinations, results, transfer certificates, smart-class digital learning content, and a multi-level competitive event system (**Competition / التنافس**) with a public live scoreboard.

Total pages (App Router)	**193** `page.js` files across 11 portals + public site
Total API routes	**42** server-side route handlers under `/app/api`
Portals / role types	**11** distinct dashboard experiences, role-gated
Firestore collections	**45+** top-level collections (see Section 7)
UI components	**102** component files (37 shadcn/Radix primitives + 65 feature components)
Shared library modules	**37** files under `/lib`
Cloud Functions	**2** deployed functions (see Section 8)
Hosting	Vercel (Next.js). Firebase is used only for Firestore, Auth, Storage & Functions — **not** Firebase Hosting.

This document is a complete architectural map intended for technical handover: every portal and page route, every API endpoint, the full data model, all environment variables (names only — no secret values), external service dependencies, and a candid account of the current security posture including items that have been addressed and items still open.

> **How this document was produced:** Generated by walking the actual repository (file tree, route handlers, Firestore/Storage rules, environment variable references in source) rather than from memory or marketing copy — every route and collection listed in Sections 5–7 was enumerated directly from the codebase on the date of writing, and reflects the codebase's current state.

2. Technology Stack

Layer	Technology	Version
Framework	Next.js (App Router)	15.5.7
UI Library	React	19.1.1
Language	TypeScript configured; most application pages are plain JS with JSDoc	5.2.2
Styling	Tailwind CSS	3.3.3
Component system	shadcn-style components in `components/ui/`, built on Radix UI primitives	various
Icons	Lucide React, React Icons	—
Client state	Zustand (with `persist` middleware)	5.0.6
Database	Firebase Firestore (client SDK)	firebase 12.0.0
Auth	Firebase Authentication + custom Firestore-backed OTP portal sessions	—
Storage	Firebase Storage	—
Server-side Firebase	firebase-admin (used only in API routes / Cloud Functions)	13.7.0
Cloud Functions	Firebase Functions v2 SDK	—
Forms & validation	React Hook Form + Zod + @hookform/resolvers	—
Toasts / notifications	Sonner	1.7.4
Document export	ExcelJS, jsPDF + jspdf-autotable, pdf-lib, html2pdf.js, xlsx, jszip, file-saver	—
Charts	Recharts	3.1.0
Media hosting	Cloudinary (uploads), proxied via `/api/proxy-image`; Firebase Storage (documents/photos)	—
Payments	Razorpay (admission & exam-fee order creation, signature verification)	2.9.6
Email	Nodemailer (SMTP, email OTP), EmailJS (browser-side contact form)	—
SMS providers	Fast2SMS, MSG91, Twilio (selectable via `SMS_PROVIDER`)	—
PWA	next-pwa (service worker, offline fallback, runtime caching)	5.6.0
Date/calendar	date-fns, react-day-picker, custom Hijri (Islamic) calendar utilities	—
Deployment	Vercel (Next.js hosting); Firebase CLI used *only* for Firestore rules/indexes deploys	—

3. Repository Structure

```
/app                      Next.js App Router — all pages and API routes
  /dashboard              All 11 portal dashboards (one sub-directory per portal)
  /api                     42 server-side route handlers
  /auth                    Login pages (per portal) + student self-registration
  /admission               Public admission application form + confirmation
  /live-score              Public real-time competition scoreboard
  /results                 Public exam results lookup
  /[other public routes]   about, contact, events, news, leaders, smart-class,
                           privacy-policy, terms-and-conditions, home-control, etc.

/components                Shared React components (102 files)
  /ui                       37 shadcn-style atomic UI primitives (Radix UI + Tailwind)
  Sidebar.jsx               Role-aware navigation menu generator
  Layout.jsx                Dashboard shell (sidebar + content + exam-timer bar)
  ProfileImage.jsx           Multi-source photo resolver (Storage/Cloudinary/proxy)
  [feature components]      AttendanceReport, HallTicket, PortalOtpVerification, etc.

/lib                        Client + shared utilities (37 files)
  authStore.js                Zustand auth store — central session state
  firebase.js                  Firebase client SDK init (Auth/Firestore/Storage/Functions/AppCheck)
  firebase-admin.js            Firebase Admin SDK init (server-only, lazy singleton)
  serverAuth.js                 API route auth guard (requireServerRole / requireCentralAdmin)
  adminCredentials.js            Env-var-sourced admin login passwords (no hardcoded fallback)
  useRoleGuard.js                Client-side route-ownership redirect guard
  madrasaHierarchy.js             District ⇄ Mekhala mapping + fuzzy matching utilities
  competitionService.js / competitionUtils.js  Competition feature CRUD + business logic
  portalAuthSecurity.js / securePortalLogin.js   Custom OTP-portal login/session security
  [27 more]                    Exam/hall-ticket/PDF/export/hijri/attendance utilities

/functions                  Firebase Cloud Functions (2 deployed functions)
/firestore.rules            Firestore security rules (single source of truth for data access)
/firestore.indexes.json     Composite index definitions
/storage.rules              Firebase Storage security rules
/docs                        Project documentation (architecture, conventions, competition
                               feature spec, and a prior infrastructure/cost audit)
/public                     Static assets, PWA manifest/service worker
/hooks, /utils, /data, /pages(vestigial), /types
```

> **Note:** a `/pages` directory exists but is vestigial — it contains only a static PWA offline fallback with no routing or data logic. There is no conflict between the legacy Pages Router and the App Router; the App Router is the only active routing system.

4. Application Architecture

4.1 Rendering & Routing Model

All dashboard and public pages use Next.js 15 App Router. The overwhelming majority of pages (~88%) are Client Components (`"use client"`) that fetch data from Firestore directly via the client SDK inside `useEffect` / `onSnapshot` , rather than through Server Components or Server Actions. This is a deliberate/inherited architectural pattern — there is effectively no server-rendering or Next.js data-caching layer between the browser and Firestore for most pages. There is no `middleware.js` anywhere in the repository; all authorization is enforced per-page (client-side, via the auth store and role guard) and per-API-route (server-side, via `lib/serverAuth.js`), never at the edge.

4.2 Authentication & Session Architecture

The platform uses **two parallel authentication mechanisms** depending on role:

A. Firebase-Auth-backed roles

`madarsa_admin, teacher, student, inspector, pariksha_board_convener, manager, central_admin`

1. `useAuthStore.initializeAuth()` subscribes to Firebase's `onAuthStateChanged` .
2. On sign-in, it sequentially queries `users → teachers → students → madarsas → managers → pariksha_board_convener` Firestore collections until a matching profile document is found.
3. The result is stored in `userData` (Zustand store), persisted to `localStorage` under the key `auth-storage` (schema version 2).
4. Login itself is brokered server-side through `/api/auth/portal-login` , which verifies credentials (Firebase Auth REST API, or a legacy plaintext-password fallback stored on the profile document), mints a Firebase custom token, and sets a custom claim `role` on the Firebase Auth user.

B. Custom-portal (non-Firebase-Auth) roles

`convener_admin, mekhala_admin` (District & Sub-Mekhala portals)

These accounts have **no Firebase Auth credential at all**. They authenticate against dedicated Firestore collections (`conveners` , `mekhala` , `districts`) via `/api/auth/portal-login` , and the client maintains a **client-trusted session** (no server-verifiable ID token) for up to 12 hours, tracked via `authSessionIssuedAt` in the persisted `userData` .

C. OTP verification layer

A phone/email OTP step (`lib/portalAuthSecurity.js` , `/api/auth/phone-otp` , `/api/auth/email-otp`) sits in front of some portal logins. Key parameters found in source:

OTP validity window	10 minutes
Phone-OTP rate limit	3 sends per phone number per 10-minute sliding window (`phone_otp_rate_limits` collection)
Password lockout	3 failed attempts → 4-hour lock (client-side, per portal+identifier)
Session length after OTP	12 hours for custom-portal roles
SMS providers	Fast2SMS / MSG91 / Twilio, selected via `SMS_PROVIDER` env var
Dev bypass code	Fixed value, gated by `NODE_ENV` and (for the email-OTP path) additionally by `NEXT_PUBLIC_ENABLE_DEV_OTP` — confirmed correctly gated and unreachable in a standard production deployment

Firestore rules explicitly deny all client read/write access to `phone_otps` , `phone_otp_rate_limits` , and `email_otps` — these are admin-SDK-only by design, which is correct.

4.3 Portal Map & Role Resolution

The authoritative routing table, taken directly from `lib/useRoleGuard.js` (the client-side guard that redirects a signed-in user away from any dashboard path it doesn't own):

Role	portalType (if applicable)	Owned base path
`central_admin` / `admin`	—	`/dashboard/dkju-control`
`madarsa_admin`	—	`/dashboard/madarsa`
`inspector`	—	`/dashboard/inspector`
`mekhala_admin`	default / `district_portal`	`/dashboard/district`
`mekhala_admin`	`mekhala_portal`	`/dashboard/mekhala-portal`
`convener_admin`	default	`/dashboard/convener`
`convener_admin`	`state_portal`	`/dashboard/state`
`manager`	—	`/dashboard/manager`

Role	portalType (if applicable)	Owned base path
`pariksha_board_convener`	—	`/dashboard/pariksha-board-convener`
`teacher`	—	`/dashboard/teacher`
`student`	—	`/dashboard/student`

Human-facing portal names map to the underlying login collection and role as follows (from `/api/auth/portal-login/route.js`):

Login page	Backing Firestore collection	Resulting role	portalType set	Redirects to
`/auth/login/dkju-control`	`users`	`central_admin`	—	`/dashboard/dkju-control`
`/auth/login/madarsa-admin`	`madarsas` (by `adminEmail`)	`madarsa_admin`	—	`/dashboard/madarsa`
`/auth/login/teacher`	`teachers`	`teacher`	—	`/dashboard/teacher`
`/auth/login/student`	`students`	`student`	—	`/dashboard/student`
`/auth/login/manager`	`managers`	`manager`	—	`/dashboard/manager`
`/auth/login/inspector`	`users` (role=inspector)	`inspector`	—	`/dashboard/inspector`
`/auth/login/pariksha-board-convener`	`pariksha_board_convener`	`pariksha_board_convener`	—	`/dashboard/pariksha-board-convener`
`/auth/login/convener`	`conveners` (or central-admin fallback)	`convener_admin`	—	`/dashboard/convener`
`/auth/login/state`	(central-admin-as-convener path)	`convener_admin`	`state_portal`	`/dashboard/state/competition`
`/auth/login/district`	`mekhala` collection	`mekhala_admin`	`district_portal`	`/dashboard/district`
`/auth/login/mekhala-admin`	`districts` collection	`mekhala_admin`	`mekhala_portal`	`/dashboard/mekhala-portal`

> **Naming quirk worth flagging to engineers taking over this code:** the login portal named **"District"** (`portal: "mekhala"` in the API) authenticates against the `mekhala` Firestore collection, while the login portal named **"Mekhala Admin"** (`portal: "mekhala-admin"`) authenticates against the `districts` collection — the collection names are swapped relative to the portal names. Both resolve to the same `mekhala_admin` role, disambiguated only by `portalType`. This is easy to misread when navigating the code for the first time and should be called out explicitly during technical handover conversations.

> **Internal documentation corrected:** the project's own `docs/architecture.md` previously described a single dual-mode page at `/dashboard/mekhala` serving both District- and Mekhala-level admins. That directory does not exist in the current codebase — the actual implementation uses two separate directories, `/dashboard/district` and `/dashboard/mekhala-portal`, as shown in the table above. The internal docs have since been corrected to match, along with the same stale path reference in `docs/conventions.md` and `docs/competition-feature.md`.

4.4 Layout, Navigation & UI Composition

Every portal page is wrapped in `<Layout>` (`components/Layout.jsx`), which renders the role-aware `<Sidebar>`, the main content area, and a bottom bar showing a live exam countdown timer where applicable. The sidebar (`components/Sidebar.jsx`) builds its menu as arrays with inline conditional insertion based on `userData.role`, `userData.portalType`, and a `compVis` visibility state that mirrors the `competition/config` Firestore document (controls whether the Competition menu item is shown per portal, without a load-time flash of the wrong state).

5. Complete Portal & Page Inventory

Every `page.js` / `page.jsx` file under `/app` is listed below, grouped by portal directory, enumerated directly from the repository file tree. Dynamic segments are shown in `[brackets]` as they appear in the routing structure.

5.1 Dashboard Portals (11 portals, 152 pages)

Central Admin Portal

Base path: `/dashboard/dkju-control` · 41 page(s)

Route	Page
`/dashboard/dkju-control/analytics`	Analytics
`/dashboard/dkju-control/applications`	Applications
`/dashboard/dkju-control/certificates`	Certificates
`/dashboard/dkju-control/competition`	Competition
`/dashboard/dkju-control/config`	Config
`/dashboard/dkju-control/districts`	Districts
`/dashboard/dkju-control/exam/[examId]`	Exam
`/dashboard/dkju-control/exam/create-admin`	Create Admin
`/dashboard/dkju-control/exam/create`	Create
`/dashboard/dkju-control/exam/data-sheet`	Data Sheet
`/dashboard/dkju-control/exam/hall-ticket-assignment`	Hall Ticket Assignment
`/dashboard/dkju-control/exam/notices`	Notices
`/dashboard/dkju-control/exam`	Exam
`/dashboard/dkju-control/exam/registrations`	Registrations
`/dashboard/dkju-control/exam/result/[examId]`	Result
`/dashboard/dkju-control/exam/result/[examId]/view`	View
`/dashboard/dkju-control/exam/timetables`	Timetables
`/dashboard/dkju-control/exam/unpublished`	Unpublished
`/dashboard/dkju-control/fix-admit`	Fix Admit
`/dashboard/dkju-control/fix-emails`	Fix Emails
`/dashboard/dkju-control/fix-temp`	Fix Temp
`/dashboard/dkju-control/inspectors`	Inspectors
`/dashboard/dkju-control/login-logs`	Login Logs
`/dashboard/dkju-control/madarsas`	Madarsas
`/dashboard/dkju-control/managers`	Managers
`/dashboard/dkju-control/mekhala`	Mekhala
`/dashboard/dkju-control/notices`	Notices
`/dashboard/dkju-control`	Dkju Control
`/dashboard/dkju-control/pariksha-board-convener`	Pariksha Board Convener
`/dashboard/dkju-control/reports/[reportId]`	Reports
`/dashboard/dkju-control/reports/attendance`	Attendance
`/dashboard/dkju-control/reports/contacts`	Contacts
`/dashboard/dkju-control/reports/manager-report`	Manager Report
`/dashboard/dkju-control/reports`	Reports
`/dashboard/dkju-control/sadar-notices`	Sadar Notices
`/dashboard/dkju-control/sadar-reviews`	Sadar Reviews
`/dashboard/dkju-control/smart-class`	Smart Class

Route	Page
`/dashboard/dkju-control/students`	Students
`/dashboard/dkju-control/style-guide`	Style Guide
`/dashboard/dkju-control/tc-requests`	Tc Requests
`/dashboard/dkju-control/teachers`	Teachers

Convener / State Portal (legacy dir)

Base path: `/dashboard/convener` · 9 page(s)

Route	Page
`/dashboard/convener/attendance`	Attendance
`/dashboard/convener/districts`	Districts
`/dashboard/convener/exam`	Exam
`/dashboard/convener/finance`	Finance
`/dashboard/convener/history`	History
`/dashboard/convener/madarsas`	Madarsas
`/dashboard/convener/notices`	Notices
`/dashboard/convener`	Convener
`/dashboard/convener/profile`	Profile

State Portal (portalType: state_portal)

Base path: `/dashboard/state` · 3 page(s)

Route	Page
`/dashboard/state/competition`	Competition
`/dashboard/state`	State
`/dashboard/state/verify-score`	Verify Score

District Portal (mekhala_admin, default)

Base path: `/dashboard/district` · 14 page(s)

Route	Page
`/dashboard/district/attendance`	Attendance
`/dashboard/district/competition`	Competition
`/dashboard/district/districts`	Districts
`/dashboard/district/exam`	Exam
`/dashboard/district/finance`	Finance
`/dashboard/district/history`	History
`/dashboard/district/madarsas`	Madarsas
`/dashboard/district/notices`	Notices
`/dashboard/district`	District
`/dashboard/district/profile`	Profile
`/dashboard/district/reports`	Reports
`/dashboard/district/students`	Students
`/dashboard/district/tc-requests`	Tc Requests
`/dashboard/district/teachers`	Teachers

Sub-Mekhala Portal (mekhala_admin, portalType: mekhala_portal)

Base path: `/dashboard/mekhala-portal` · 9 page(s)

Route	Page
`/dashboard/mekhala-portal/competition`	Competition
`/dashboard/mekhala-portal/exam`	Exam
`/dashboard/mekhala-portal/history`	History
`/dashboard/mekhala-portal/madarsas`	Madarsas
`/dashboard/mekhala-portal/notices`	Notices
`/dashboard/mekhala-portal`	Mekhala Portal
`/dashboard/mekhala-portal/profile`	Profile
`/dashboard/mekhala-portal/students`	Students
`/dashboard/mekhala-portal/teachers`	Teachers

Sadar (Madarsa) Portal

Base path: `/dashboard/madarsa` · 20 page(s)

Route	Page
`/dashboard/madarsa/applications`	Applications
`/dashboard/madarsa/assignments`	Assignments
`/dashboard/madarsa/attendance`	Attendance
`/dashboard/madarsa/bulk-register`	Bulk Register
`/dashboard/madarsa/competition`	Competition
`/dashboard/madarsa/contacts`	Contacts
`/dashboard/madarsa/exam/exams`	Exams
`/dashboard/madarsa/exam/exams/results/[examId]`	Results
`/dashboard/madarsa/exam/notices`	Notices
`/dashboard/madarsa/exam`	Exam
`/dashboard/madarsa/exam/students`	Students
`/dashboard/madarsa/exam/timetables`	Timetables
`/dashboard/madarsa/managers`	Managers
`/dashboard/madarsa/mekhala-notices`	Mekhala Notices
`/dashboard/madarsa/notices`	Notices
`/dashboard/madarsa`	Madarsa
`/dashboard/madarsa/students`	Students
`/dashboard/madarsa/tc-requests`	Tc Requests
`/dashboard/madarsa/teachers`	Teachers
`/dashboard/madarsa/thafteesh-review`	Thafteesh Review

Pariksha Board Portal

Base path: `/dashboard/pariksha-board-convener` · 16 page(s)

Route	Page
`/dashboard/pariksha-board-convener/certificates`	Certificates
`/dashboard/pariksha-board-convener/data-sheet`	Data Sheet
`/dashboard/pariksha-board-convener/exams/[examId]/unregistered`	Unregistered
`/dashboard/pariksha-board-convener/exams/create`	Create
`/dashboard/pariksha-board-convener/exams`	Exams
`/dashboard/pariksha-board-convener/hall-tickets`	Hall Tickets
`/dashboard/pariksha-board-convener/marks-entry`	Marks Entry
`/dashboard/pariksha-board-convener`	Pariksha Board Convener

Route	Page
`/dashboard/pariksha-board-convener/profile`	Profile
`/dashboard/pariksha-board-convener/registrations`	Registrations
`/dashboard/pariksha-board-convener/results/[examId]`	Results
`/dashboard/pariksha-board-convener/results/[examId]/view`	View
`/dashboard/pariksha-board-convener/results`	Results
`/dashboard/pariksha-board-convener/students`	Students
`/dashboard/pariksha-board-convener/unpublished`	Unpublished
`/dashboard/pariksha-board-convener/unregistered-students`	Unregistered Students

Inspector Portal

Base path: `/dashboard/inspector` · 15 page(s)

Route	Page
`/dashboard/inspector/attendance`	Attendance
`/dashboard/inspector/contacts`	Contacts
`/dashboard/inspector/history`	History
`/dashboard/inspector/inspections/[inspectionId]`	Inspections
`/dashboard/inspector/inspections/create`	Create
`/dashboard/inspector/inspections`	Inspections
`/dashboard/inspector/inspections/upload`	Upload
`/dashboard/inspector/madarsas/[madarsaId]/inspect`	Inspect
`/dashboard/inspector/madarsas/[madarsaId]`	Madarsas
`/dashboard/inspector/madarsas`	Madarsas
`/dashboard/inspector/management`	Management
`/dashboard/inspector/notices`	Notices
`/dashboard/inspector`	Inspector
`/dashboard/inspector/records`	Records
`/dashboard/inspector/reports`	Reports

Manager Portal

Base path: `/dashboard/manager` · 6 page(s)

Route	Page
`/dashboard/manager/documents`	Documents
`/dashboard/manager`	Manager
`/dashboard/manager/reports`	Reports
`/dashboard/manager/students`	Students
`/dashboard/manager/tc-requests`	Tc Requests
`/dashboard/manager/teachers`	Teachers

Teacher Portal

Base path: `/dashboard/teacher` · 8 page(s)

Route	Page
`/dashboard/teacher/attendance`	Attendance
`/dashboard/teacher/materials`	Materials
`/dashboard/teacher/notices`	Notices
`/dashboard/teacher`	Teacher
`/dashboard/teacher/profile`	Profile

Route	Page
`/dashboard/teacher/smart-class`	Smart Class
`/dashboard/teacher/students`	Students
`/dashboard/teacher/tc-requests`	Tc Requests

Student Portal

Base path: `/dashboard/student` · 11 page(s)

Route	Page
`/dashboard/student/assignments`	Assignments
`/dashboard/student/exam/notices`	Notices
`/dashboard/student/exam`	Exam
`/dashboard/student/exam/registrations`	Registrations
`/dashboard/student/exam/timetables`	Timetables
`/dashboard/student/notices`	Notices
`/dashboard/student`	Student
`/dashboard/student/profile`	Profile
`/dashboard/student/results`	Results
`/dashboard/student/smart-class`	Smart Class
`/dashboard/student/tc-request`	Tc Request

5.2 Public Site, Auth & Utility Pages (41 pages)

Includes the public marketing/informational site, all per-portal login pages, the public admission funnel, and a small number of internal test/utility pages. `/test-camera` (a thin, unreferenced wrapper around the shared `CameraCapture` component with zero unique logic) has since been removed as dead code; `/auth-test` and `/test-count` were reviewed and kept in place, since both exercise real, still-in-use application logic rather than disposable scaffolding.

Route	Page
`/about`	About
`/admin/style-guide`	Style Guide
`/admission/confirmation`	Confirmation
`/admission/form`	Form
`/auth-test`	Auth Test
`/auth/login/convener`	Convener
`/auth/login/district`	District
`/auth/login/dkju-control`	Dkju Control
`/auth/login/inspector`	Inspector
`/auth/login/madarsa-admin`	Madarsa Admin
`/auth/login/manager`	Manager
`/auth/login/mekhala-admin`	Mekhala Admin
`/auth/login/pariksha-board-convener`	Pariksha Board Convener
`/auth/login/state`	State
`/auth/login/student`	Student
`/auth/login/teacher`	Teacher
`/auth/register/student`	Student
`/brand-guide`	Brand Guide
`/central-access`	Central Access
`/coming-soon`	Coming Soon

Route	Page
`/contact`	Contact
`/dkimvb`	Dkimvb
`/dkju-axis`	Dkju Axis
`/duplicates/applications`	Applications
`/events`	Events
`/home-control/login`	Login
`/home-control`	Home Control
`/leaders`	Leaders
`/live-score`	Live Score
`/master/register`	Register
`/mobile-upload`	Mobile Upload
`/new-home`	New Home
`/news`	News
`/offline`	Offline
`/`	Home
`/privacy-policy`	Privacy Policy
`/results`	Results
`/smart-class`	Smart Class
`/terms-and-conditions`	Terms And Conditions
`/test-count`	Test Count
`/unauthorized`	Unauthorized

6. API Route Inventory

42 server-side route handlers under `/app/api`, all running on the Node.js runtime (not Edge), all using the Firebase Admin SDK for any Firestore/Auth/Storage access. Grouped by function below. The **Auth** column reflects whether the route itself enforces a server-verified identity/role check (via `requireServerRole` / `requireCentralAdmin` in `lib/serverAuth.js`, which validates a Firebase ID token and checks the caller's role) — a more detailed, line-by-line version of this audit exists in `docs/audits/FIREBASE_COST_AUDIT.md` (§5) already in the repository.

6.1 Authentication

Route	Purpose	Auth
`/api/auth/portal-login`	Central login broker for all 11 portal types — verifies credentials, mints Firebase custom token	Self-contained
`/api/auth/resolve-role`	Resolves role from a Firebase ID token on session bootstrap	Verified
`/api/auth/phone-otp`	Send/verify phone OTP — hashed storage, rate-limited	Self-contained (OTP is the check)
`/api/auth/email-otp`	Send/verify email OTP via SMTP	Self-contained (OTP is the check)
`/api/validate-credential`	Validates an email/password pair via Firebase Auth REST API	Self-contained
`/api/validate-brand-password`	Password check against admin passwords (env-sourced, see §10.2) plus plaintext `password` fields across 6 collections	Self-contained (this IS the auth check)
`/api/security/firebase-config`	Serves the public Firebase client config as a JS snippet (runtime override path)	Public config, by design
`/api/security/client-context`	Header-based bot-detection heuristics	Is the check itself

6.2 Admin Operations (Central Admin)

Route	Purpose	Auth
`/api/admin/certificates-data`	Filterable student/certificate lookup	Verified
`/api/admin/create-manager`	Create/update a manager account (Auth + Firestore)	Verified
`/api/admin/delete-user`	Delete a Firebase Auth user + Firestore doc	Verified (central admin only)
`/api/admin/get-admissions`	Admissions lookup by date range	Verified
`/api/admin/settings`	Update global app settings	Verified
`/api/admin/visitor-reports`	Login/visitor logs (up to 5,000 rows)	Verified
`/api/admin/check-managers`	List all manager accounts	No guard — low-sensitivity read, flagged for future hardening

6.3 Exams & Payments

Route	Purpose	Auth
`/api/exams/update-hall-tickets`	Bulk hall-ticket field update (batched writes)	Verified
`/api/create-exam-order`	Create a Razorpay order for exam registration fees	Verified
`/api/create-order`	Create a Razorpay order for admission fees (older code path)	Possibly superseded — worth confirming which is live
`/api/verify-payment`	Verify Razorpay payment signature (HMAC) and capture	HMAC signature is the check

6.4 Public / Content

Route	Purpose	Auth
`/api/public/admission`	Look up one admission application by number	Public, by design
`/api/public/admission-number`	Sequential admission number generator (counter transaction)	Public, by design
`/api/public/madarsas`	Public madrasa directory (dropdowns/forms)	Public, by design
`/api/public/search-person`	Public person search (name/address/phone-last-3/DOB)	Public PII search — unchanged, out of scope for this pass
`/api/news`, `/api/news/[id]`	Public news feed / admin create-edit-delete	GET public, write verified
`/api/hero-images`, `/api/hero-images/[id]`	Homepage hero image list/add/delete	GET public, write verified

Route	Purpose	Auth
`/api/proxy-image`	Image proxy for Cloudinary/Storage photos (3-tier fallback)	Public, by design
`/api/iframe-proxy`	Reverse proxy for an external government site (awqaf.gov.ae), also wired as the site-wide catch-all rewrite in `next.config.js`	Open `?url=` param — unchanged, out of scope for this pass
`/api/student/madarsa-details`	Madrasa contact info for the student portal	Verified
`/api/inspector/madarsas`	5-minute cached madrasa list for inspector portal	No guard, but read-only + cached

6.5 Competition, Mobile Upload & Misc.

Route	Purpose	Auth
`/api/mobile-upload/session`	Create/poll a short-lived (10-min) mobile-upload session for inspectors	POST verified, GET session-ID-gated
`/api/mobile-upload/upload`	Accept a phone-camera file, save to Storage, link to inspection	Unguessable session ID only

6.6 Debug / Seed / Demo Endpoints — Remediated

The following endpoints are development/debugging aids left in the deployed API surface. None required authentication and several were destructive. **Status: all 9 have since been gated behind `requireCentralAdmin()`** — the same server-verified role guard already used by the rest of the admin API surface (Section 6.2). Each now requires a valid central-admin Firebase ID token (`Authorization: Bearer <token>`) or returns 401/403; a full production build (`next build` , exit code 0) was run after the change, and again after every subsequent change in this document's remediation history, to confirm nothing else in the app was affected. The code paths themselves are unchanged — this was intentional, since none of the nine are referenced by any frontend page and removing the logic outright was judged higher-risk than simply closing public access to it.

Route	Behavior	Status
`/api/debug-districts`	Dumps all districts + mekhalas	Fixed — central-admin only
`/api/debug-madarsa`	Dumps 5 raw madrasa + 5 raw student documents	Fixed — central-admin only
`/api/debug-matching`	Dumps district/mekhala name-matching results	Fixed — central-admin only
`/api/debug-pothencode`	One-off debugging query for a specific mekhala	Fixed — central-admin only
`/api/inspect-student`	Returns the first 3 raw student documents, no parameters	Fixed — central-admin only
`/api/clean-unknown`	Deletes documents across 4 competition collections	Fixed — central-admin only
`/api/seed-competition`	Wipes 4 competition collections and reseeds fake data	Fixed — central-admin only
`/api/simulate-live-score`	Injects one fake high score for a random real student	Fixed — central-admin only
`/api/init-events`	One-time competition-events seeding endpoint	Fixed — central-admin only

Remaining recommendation: once confirmed nothing depends on them, deleting `clean-unknown` , `seed-competition` , and `simulate-live-score` outright (rather than just guarding them) remains the cleanest long-term option, since they have no legitimate role in a live production system.

7. Data Model — Firestore Collections

Single Cloud Firestore database. Collections are grouped below by domain; access rules for each are enforced in `firestore.rules` (see Section 11 for the full security-rule model).

7.1 Identity & Institution Collections

Collection	Purpose
`users`	Admin-tier accounts: central_admin, admin, inspector, home_admin
`madarsas`	Individual madrasa (Sadar) records; `adminEmail` links to Firebase Auth. Uses field names `mekhala` / `district` (not `mekhalaName` / `districtName`) — a known naming quirk callers must handle
`students`	Student profiles — admission number, DOB, madrasa/mekhala/district linkage
`teachers`	Teacher profiles
`managers`	Manager accounts, linked to a madrasa
`pariksha_board_convener`	Exam board convener accounts
`districts`	District-tier metadata (also used as the login collection for the "Mekhala Admin" portal — see §4.3 quirk)
`mekhala`	Mekhala-tier metadata (also used as the login collection for the "District" portal — see §4.3 quirk)
`conveners`	State convener records

7.2 Academics — Exams, Attendance, Results

Collection	Purpose
`exams`	Exam definitions
`examRegistrations`	Student exam registrations
`hallTickets/{examId}/students/{studentId}`	Subcollection — per-exam hall tickets
`hallTicketCounters`	Sequencing counters for hall-ticket numbers
`exammarks`	Marks per exam (restricted to exam admins)
`publishedresults` , `results`	Published exam results (public read)
`resultSubjectTemp`	Working/temp table during result compilation
`examNotices` , `examTimetables`	Exam notices & timetables (public read)
`attendances`	Daily attendance records
`attendance_audit_logs`	Audit trail for attendance edits
`tc_requests`	Transfer-certificate requests

7.3 Admissions, Notices & Content

Collection	Purpose
`admissions`	Admission applications (public create; unauthenticated applicants)
`counters`	Sequential-number counters (e.g. admission numbers)
`notices` , `centralNotices` , `noticeAdmin`	Institutional notices (public read)
`news` , `events` , `carousel` , `marquee` , `marqueeTexts` , `homepageConfig`	Public homepage content
`smartClassSyllabus` , `smartClassContents`	Public smart-class digital learning content
`settings`	Global app settings (public read)
`brand_assets`	Branding assets (logos etc.)
`directory_contacts`	Central-admin-only contact directory

7.4 Inspection Workflow

Collection	Purpose
`inspections`	Inspector visit/inspection records
`inventory`	Madrasa inventory records captured during inspection
`sadarReviews`	Sadar-level inspection reviews
`mobileUploadSessions`	Short-lived sessions for phone-camera inspection uploads

7.5 Competition (التنافس) Collections

Collection	Purpose
`competition`	Single document `config` — categories, DOB cutoffs, per-portal visibility flags
`competition_events`	Event definitions (name, category, individual/group, participants required)
`competition_registrations`	Sadar-level student registrations per event
`competition_mekhala_marks`	Marks entered at Mekhala level
`competition_district_marks`	Marks entered at District level
`competition_state_marks`	Marks entered at State level, with verification workflow
`competition_results`	Published/verified results (public read — powers the live scoreboard)

Data flows strictly upward: **Sadar** → **Mekhala** → **District** → **State** → **Verify** → **Public Results**. Grade scale: 3 marks out of 50 each (0–150 total) → A+ (135–150) / A (115–134) / B+ (95–114) / B (75–94) / C (55–74) / D (0–54). Full specification in `docs/competition-feature.md` (already in the repository).

7.6 Auth & Security Collections (no client access)

Collection	Purpose
`phone_otps`, `phone_otp_rate_limits`, `email_otps`	OTP session/rate-limit state. Firestore rules explicitly deny all client read/write — admin-SDK-only.
`LoginLogs`	Per-session login/logout audit trail with duration tracking

8. Firebase Cloud Functions

Two functions deployed from `functions/index.js` (the `main` entry point per `functions/package.json`). Note: `functions/src/index.js` is a stale, unused duplicate left in the repository and can be safely removed.

Function	Trigger	Purpose
`deleteInspectorUser`	HTTPS Callable (`onCall`)	Deletes a Firebase Auth user + their Firestore `users` doc. Server-side checks that the caller's own `users` doc has role `central_admin` or `admin` before proceeding.
`syncStudentToHallTickets`	Firestore trigger — `onDocumentUpdated("students/{studentId}")`	Whenever a student document is edited, propagates key fields (name, gender, guardian phone, class, admission number, photo) into every hall-ticket subcollection copy of that student across all exams, via a `collectionGroup` query and batched writes (450/batch).

9. External Services & Third-Party Dependencies

Service	Used for	Integration point
Firebase (Google Cloud)	Firestore database, Authentication, Cloud Storage, Cloud Functions, App Check	`lib/firebase.js` (client), `lib/firebase-admin.js` (server)
Vercel	Application hosting (Next.js) — the app is **not** hosted on Firebase Hosting	Deployment platform
Cloudinary	Image upload/hosting for profile pictures and documents	`lib/cloudinary.js`, proxied through `/api/proxy-image`
Razorpay	Payment processing for admission fees and exam registration fees	`/api/create-order`, `/api/create-exam-order`, `/api/verify-payment`
SMS gateway (Fast2SMS / MSG91 / Twilio)	Phone OTP delivery — provider selectable at runtime	`/api/auth/phone-otp`
SMTP (custom host)	Email OTP delivery via Nodemailer	`/api/auth/email-otp`
EmailJS	Browser-side contact-form email delivery (no server round-trip)	`components` contact form
Google reCAPTCHA v3	Firebase App Check attestation provider	`lib/firebase.js` (production only)
Google Fonts	Web fonts, cached via PWA service worker	`next.config.js` runtime caching rules
awqaf.gov.ae (external gov. site)	Reverse-proxied content — see Section 11 for the associated note	`/api/iframe-proxy`

10. Environment Variables Reference

Variable names only — no values are reproduced in this document, consistent with standard secret-handling practice. Actual values must be transferred to the client through a secure channel (a password manager shared vault, or the hosting platform's environment variable UI directly), never through this document, email, or chat.

10.1 Firebase (client + server)

Variable	Scope	Purpose
`NEXT_PUBLIC_FIREBASE_API_KEY`	Client	Firebase Web API key
`NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN`	Client	Firebase Auth domain
`NEXT_PUBLIC_FIREBASE_PROJECT_ID`	Client	Firebase project identifier
`NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET`	Client	Storage bucket name
`NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID`	Client	FCM sender ID
`NEXT_PUBLIC_FIREBASE_APP_ID`	Client	Firebase app ID
`NEXT_PUBLIC_FIREBASE_APPCHECK_SITE_KEY`	Client	reCAPTCHA v3 site key for App Check
`FIREBASE_API_KEY` / `FIREBASE_AUTH_DOMAIN` / `FIREBASE_PROJECT_ID` / `FIREBASE_STORAGE_BUCKET` / `FIREBASE_MESSAGING_SENDER_ID` / `FIREBASE_APP_ID` / `FIREBASE_APPCHECK_SITE_KEY`	Server	Server-side mirrors of the above, read by API routes that build the client config server-side
`FIREBASE_PRIVATE_KEY`	Server (secret)	Service-account private key for Admin SDK
`FIREBASE_CLIENT_EMAIL`	Server	Service-account client email for Admin SDK
`FIREBASE_SERVICE_ACCOUNT_KEY`	Server (secret)	Fallback: full service-account JSON, used if the two above are absent

10.2 Admin Access Control

Variable	Purpose
`CENTRAL_ADMIN_EMAILS`	Comma-separated allowlist of emails granted central-admin access, in addition to a hardcoded fallback list in source (see §11)
`ADMIN_PW_YOURLETTERS2ME_GMAIL` / `ADMIN_PW_YOURLETTERS2ME_LIVE` / `ADMIN_PW_SUDARSANAN` / `ADMIN_PW_MAKEWITHUS` / `ADMIN_PW_KRRISH`	**Required.** Per-account admin login passwords. `lib/adminCredentials.js` reads these with no hardcoded fallback — if unset, that specific account cannot log in via this path (fails safely, logged clearly server-side; does not affect any other account or portal).
`ADMIN_PW_ADMINISTRATOR_DKJU` / `ADMIN_PW_MWUS` / `ADMIN_PW_DEV_MWUS` / `ADMIN_PW_DEVELOPER_MWUS`	**Required.** Same purpose as above, for the remaining admin/home-portal accounts.

> **Action needed from you:** add the nine `ADMIN_PW_*` variables above to `.env` and to the Vercel production project with the current values for each account (the repo owner has these; this document and the engineering work behind it never stored or invented any of them). Until each is set, that specific account cannot log in via this path — every other account and every other portal is unaffected. This document's author did not modify `.env` or any hosting dashboard directly, per this project's standing rule that only the repository owner edits environment/secret configuration.

10.3 Payments (Razorpay)

Variable	Purpose
`NEXT_PUBLIC_RAZORPAY_KEY_ID`	Client-side Razorpay checkout key
`RAZORPAY_KEY_ID`	Server-side Razorpay key
`RAZORPAY_KEY_SECRET`	Server-side Razorpay secret (used for order creation + signature verification)

10.4 Communications (Email / SMS)

Variable	Purpose
`NEXT_PUBLIC_EMAILJS_SERVICE_ID` / `NEXT_PUBLIC_EMAILJS_TEMPLATE_ID` / `NEXT_PUBLIC_EMAILJS_PUBLIC_KEY`	Browser-side EmailJS contact-form configuration

Variable	Purpose
`SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASS`	SMTP credentials for email OTP delivery
`SMS_PROVIDER`	Selects active SMS gateway: `fast2sms` / `msg91` / `twilio`
`FAST2SMS_API_KEY`	Fast2SMS gateway credential
`MSG91_API_KEY` / `MSG91_SENDER` / `MSG91_TEMPLATE_ID`	MSG91 gateway credentials
`TWILIO_ACCOUNT_SID` / `TWILIO_AUTH_TOKEN` / `TWILIO_FROM_NUMBER`	Twilio gateway credentials

10.5 Miscellaneous

Variable	Purpose
`NEXT_PUBLIC_QR_BASE_URL`	Base URL used when generating QR codes (e.g. hall tickets)
`NEXT_PUBLIC_SITE_URL`	Canonical site URL for absolute-link generation
`NEXT_PUBLIC_ENABLE_DEV_OTP`	Second, redundant gate on the email-OTP dev bypass (also requires `NODE_ENV` to not be production). Recommend leaving unset in production
`NODE_ENV`	Standard Node environment flag. Recommend **not** setting this manually in Vercel at all — let Vercel manage it automatically
`TARGET_AUTH_UID` / `TARGET_AUTH_PASSWORD`	Appear to back a one-off maintenance/migration script — confirm still needed, remove if not

11. Security Implementation & Findings

This section documents what is implemented today, followed by a candid list of open items for the incoming technical team to prioritize.

11.1 Firestore Security Rules — Model

Access control lives centrally in `firestore.rules`, built around a small set of reusable predicate functions:

- `signedIn()` — is there an authenticated Firebase user
- `hasUserRole(role)` / `hasAnyUserRole(roles)` — checks the custom-claim `role` on the ID token first, then falls back to reading the caller's own `users/{uid}` document
- `isCentralAdmin()` — role is `central_admin` or `admin`
- `isStaff()` — any of: madarsa_admin, teacher, inspector, mekhala_admin, convener_admin, pariksha_board_convener, manager, or central admin
- `isExamAdmin()` — central admin or pariksha_board_convener
- `isSelf(docId)` — the caller's UID matches the document ID

Notable hardening already present:

- `roleUnchanged()` — a user updating their own `users` document cannot change their own `role` field.
- `selfCreateSafeUserDoc()` / `staffCreateSafeUserDoc()` — a self-registering student (or staff creating a student doc) cannot smuggle in `central_admin`, `admin`, `customClaims`, or `permissions` fields.
- `isSafeOwnStudentUpdate()` — students editing their own record via `isSelf` / `isOwnEmailRecord` may only touch an explicit allowlist of fields (name, address, DOB, guardian name, profile picture) — cannot touch grades, status, or admission data.
- `isSafeLoginSessionUpdate()` — a user updating their own `LoginLogs` entry (heartbeat/session-close) may only touch `durationSeconds`, `lastSeenAt`, `lastSeenAtClient`, `status`, with type/range validation on each.
- OTP collections (`phone_otps`, `phone_otp_rate_limits`, `email_otps`) are hard-denied to all client access — admin-SDK-only, correctly enforced.
- A default-deny catch-all (`match /{document=**} { allow read, write: if false; }`) closes off any collection not explicitly matched above it.

11.2 Firebase Storage Security Rules

- General read access requires authentication; write access requires authentication **and** enforces a 25 MB size cap and an explicit content-type allowlist (images, PDF, Office formats) — blocks arbitrary executable uploads.
- Two narrow, deliberate unauthenticated-write exceptions: admission-form profile photos (5 MB cap, images only) and mobile QR inspection uploads (10 MB cap, images/PDF only) — both scoped to specific path prefixes, not a blanket allowance.

11.3 HTTP Security Headers

Set globally in `next.config.js`: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (relaxed to `SAMEORIGIN` only for the iframe-proxy route), `Referrer-Policy: strict-origin-when-cross-origin`, a scoped `Permissions-Policy`, and `Strict-Transport-Security` (1-year max-age, includeSubDomains). `poweredByHeader` is disabled and production source maps are turned off.

11.4 App Check

Firebase App Check with the ReCaptchaV3 provider is initialized client-side, but **only when** `NODE_ENV === "production"` and a site key is configured — confirm this is actually enforced (not just initialized) on the Firebase Console side for the collections that matter most.

11.5 Open Findings

> **Custom-portal sessions (District/Sub-Mekhala/State/Convener) are not server-verifiable.** Unlike Firebase-Auth-backed roles, these roles' 12-hour sessions rely on a client-trusted Zustand/localStorage flag rather than a Firebase ID token the server can independently verify on each request. Firestore rules for the underlying collections still apply (custom claims/role checks), but any server-side logic that trusts `userData` from the client without an independent role check should be reviewed. This item is intentionally out of scope for the current remediation pass — flagged for a future, carefully-tested migration to real Firebase Auth custom tokens (the codebase already mints custom tokens for other roles via the same login endpoint, so the pattern exists and could be extended).

> **Public PII search endpoint and the iframe-proxy catch-all.** `/api/public/search-person` (unauthenticated PII lookup) and `/api/iframe-proxy` (open `?url=` param, wired as the site-wide 404 fallback) were flagged in the original audit and remain unchanged — both are intentionally out of scope for the current remediation pass. See `docs/audits/FIREBASE_COST_AUDIT.md` §7/§14 for the full detail on each.

A deeper, file-and-line-level version of the read-path and API-route audit exists in the repository at `docs/audits/FIREBASE_COST_AUDIT.md` and has been kept up to date alongside this document — it also covers Firestore read-cost patterns (unbounded queries, N+1 lookups) that, while not security issues, affect both cost and scalability post-handover.

12. PWA, Offline Support & Deployment

- **Service worker:** `next-pwa`, registered in production only (`disable: NODE_ENV === "development"`), with `skipWaiting` enabled.
- **Runtime caching:** Google Fonts (CacheFirst, 1 year), images by extension (CacheFirst, 30 days).
- **Offline fallback:** static page at `/offline` (App Router) and a vestigial `pages/_offline.js`.
- **Hosting:** Vercel — confirmed via `.vercel/project.json`. Firebase Hosting is **not** used; per project convention, Firebase CLI is only ever invoked for `firestore:rules` and `firestore:indexes` deploys, never Hosting or Functions from a general workstation.
- **Images:** Next.js image optimization is disabled (`images: { unoptimized: true }`) — images are served as-is or via the Cloudinary/Storage proxy.

This document was generated directly from the DKJU repository (file structure, route handlers, Firestore/Storage security rules, and environment-variable references in source) for the purpose of technical handover, and reflects the codebase's current state. No secret values, API keys, passwords, or student/staff personal data are included anywhere in this document. `.env` and hosting-platform environment configuration are not modified as part of producing this document, per this project's standing rule that only the repository owner edits environment/secret configuration.